

Assignment No 3:

Implement Greedy search algorithm for any of the following application: I. Selection Sort II. Minimum Spanning Tree III. Single-Source Shortest Path Problem IV. Job Scheduling Problem V. Prim's Minimal Spanning Tree Algorithm VI. Kruskal's Minimal Spanning Tree Algorithm VII. Dijkstra's Minimal Spanning Tree Algorithm

A **Greedy Search Algorithm** makes locally optimal choices at each step with the hope of finding a global optimum. It does not always guarantee the best solution but is efficient for many problems.

Steps of the Greedy Algorithm:

- ✓ **Sort or prioritize choices** based on some criteria (e.g., minimum weight, earliest deadline).
- ✓ **Select the best available option** (locally optimal choice).
- ✓ **Check feasibility** (whether it satisfies constraints like no cycles in an MST).
- ✓ **Repeat** until the solution is formed.

Examples of Greedy Search Algorithms are:

- I. Selection Sort
- II. Minimum Spanning Tree
 - ✓ Prim's Minimal Spanning Tree Algorithm
 - ✓ Kruskal's Minimal Spanning Tree Algorithm
- III. Single-Source Shortest Path Problem
 - ✓ Dijkstra's Minimal Spanning Tree Algorithm
- IV. Job Scheduling Problem

I Selection Sort:

Selection Sort follows the Greedy approach by repeatedly selecting the smallest element and placing it in its correct position. It ensures that the list is sorted step-by-step, making locally optimal choices.

Steps of Selection Sort (Greedy Approach)

1. Find the **minimum element** in the unsorted part of the array.
2. Swap it with the **first element** of the unsorted part.
3. Move the boundary between sorted and unsorted parts one step forward.
4. Repeat until the entire array is sorted.

Selection Sort works by **repeatedly finding the smallest element** from the unsorted part of the list and moving it to the sorted part.

Let's take an example:

Unsorted Array: [64, 25, 12, 22, 11]

Time Complexity

- **Best case (Already Sorted):** $O(n^2)$
- **Worst case (Reversed Order):** $O(n^2)$
- **Average case:** $O(n^2)$

Python code:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i # Assume the first element is the minimum
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]: # Find the minimum element
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i] # Swap

# Example Usage
arr = [64, 25, 12, 22, 11]
print("Original array:", arr)
selection_sort(arr)
print("Sorted array:", arr)
```

Example: Greedy Algorithm for Kruskal's MST (Minimal Spanning Tree)

Kruskal's algorithm follows the **greedy strategy** by:

- ✓ **Sorting edges by weight** (greedy step).
- ✓ **Adding the smallest edge** that does not form a cycle.
- ✓ **Stopping when all vertices are connected.**